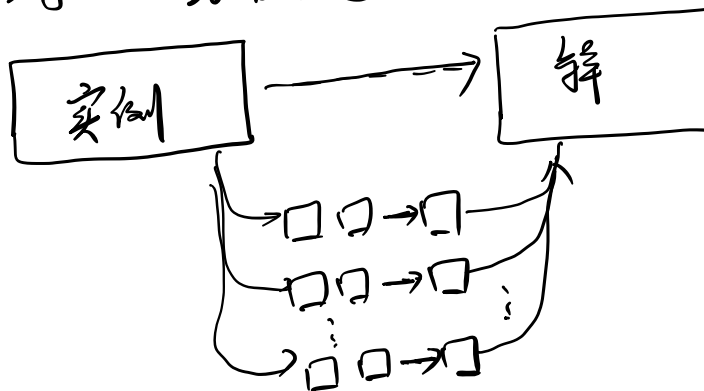


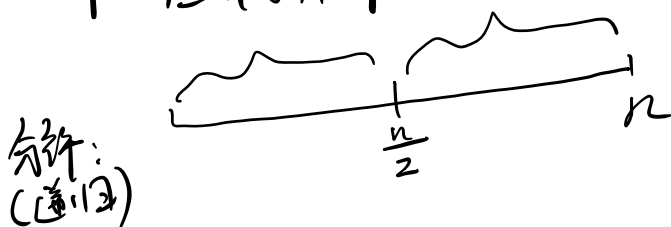
LN9. 分治算法 (Divide & Conquer), 最近点对

* 分治策略: 分而治之



- ① 分解为更小的实例.
(与原题同一类)
- ② 对每个实例递归求解.
(递归)
- ③ 合并子问题的解

例: 归并排序



合并: $O(n)$

时间复杂度: $T(n) = 2T(\frac{n}{2}) + cn = O(n \log n)$

Pf. 不失一般性, 设 $n = 2^k$:

$$T(n) = 2 \left[2T\left(\frac{n}{4}\right) + c \cdot \frac{n}{2} \right] + cn$$

$$= 2^2 T\left(\frac{n}{2^2}\right) + 2cn$$

?

$$= 2^k T(1) + k \cdot cn$$

$$= n \cdot T(1) + cn \log n$$

$$= O(n \log n)$$

* 最近点对问题.

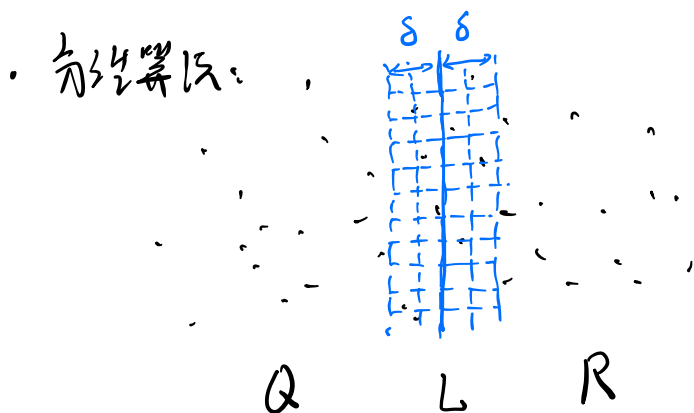
问题定义: 二维平面, 由 n 个点构成的集合 P

求: $P, P' \in P, d(P, P') = \min_{\substack{P_i, P_j \in P \\ P_i \neq P_j}} d(P_i, P_j)$

$d(\cdot, \cdot)$: 欧氏距离.

朴素算法: $C_n^2 = O(n^2)$ 复杂度.

特例: 排成一列.
但乱序给出. 首先排序再相邻比较: $O(n \log n)$



Q : L 左边的点 $\Rightarrow \min d(Q)$ 的最小值. 记为 δ .

R : L 右边的点 $\Rightarrow \min d(R)$

Lemma 1. 每个 $\frac{\delta}{2} \times \frac{\delta}{2}$ 的方格包含 ≤ 1 个点

Lemma 2. 若 $d(P_i, P_j) < \delta$. 则 P_i, P_j 全在现在这层的 3 排 $\frac{\delta}{2} \times \frac{\delta}{2}$ 内.

法①: 对 L 左右 2δ 范围内的点.

按 y 排序.

则每个点上下各 11 个点.



改②: 对L左右28范围内点上.

左右分别排序. 每个点各对面上, 向下移 6 个点.

• 算法复杂度: $T(n) = 2T(\frac{n}{2}) + cn = O(n \log n)$

LN10. 分治: 大数乘法. 矩阵乘法

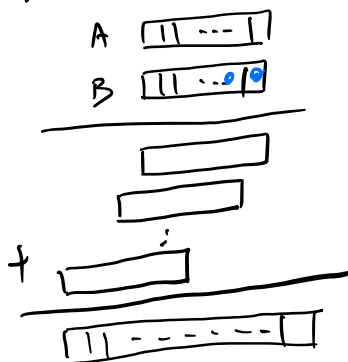
* 大数乘法: $A: \boxed{11 \dots 1} \quad n \text{ 位}$
 $B: \boxed{11 \dots 1}$

① 加法: A, B .

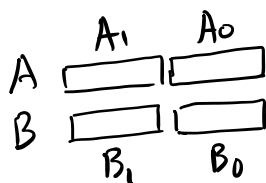
$$C = A + B \quad O(n)$$

② 乘法: $C = A \cdot B$

$$O(n^2)$$



• 分治法:



$$A = A_1 \cdot 10^{\frac{n}{2}} + A_0$$

$$B = B_1 \cdot 10^{\frac{n}{2}} + B_0$$

$$\begin{aligned} A \cdot B &= (A_1 \cdot 10^{\frac{n}{2}} + A_0)(B_1 \cdot 10^{\frac{n}{2}} + B_0) \\ &= \underline{A_1 B_1} 10^n + \underline{(A_1 B_0 + A_0 B_1)} 10^{\frac{n}{2}} + \underline{A_0 B_0} \end{aligned}$$

复杂度 $T(n) = 4T(\frac{n}{2}) + cn$

$a=4, b=2, d=1. \quad a > b^d \Rightarrow O(n^{\log_b a})$

$$T(n) = O(n^{\log_2 4}) = O(n^2)$$

• 更快的分治算法 (高斯加速)

$$\begin{cases} \textcircled{1} (A_1 + A_0)(B_1 + B_0) = A_1 B_1 + \underline{A_0 B_1 + A_1 B_0} + A_0 B_0 \\ \textcircled{2} A_1 B_1 \\ \textcircled{3} A_0 B_0 \end{cases}$$

$$\begin{aligned} A \cdot B &= (A_1 10^{\frac{n}{2}} + A_0)(B_1 10^{\frac{n}{2}} + B_0) \\ &= \underbrace{A_1 B_1}_{\textcircled{2}} 10^n + \underbrace{(A_1 B_0 + A_0 B_1)}_{\textcircled{1} - (\textcircled{2} + \textcircled{3})} 10^{\frac{n}{2}} + \underbrace{A_0 B_0}_{\textcircled{3}} \end{aligned}$$

算法方程: $T(n) = 3T(\frac{n}{2}) + cn$

$a=3, b=2, d=1 \quad a > b^d$

$$T(n) = O(n^{\log_2 3}) \approx O(n^{1.73})$$

* 矩阵乘法 (Strassen). $A, B: n \times n$

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \quad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} \Rightarrow C = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}$$

① 直接乘, 复杂度 = $O(n^3)$

$$\begin{bmatrix} A \end{bmatrix} \begin{bmatrix} B \end{bmatrix} = \begin{bmatrix} C \end{bmatrix}_{n \times n}$$

$\uparrow$
 $O(n)$

② 分治, 按分块直接乘, 复杂度 = $O(\quad)$

递归: $T(n) = aT(\frac{n}{b}) + c \cdot n^d$

$$T(n) = \begin{cases} a > b^d & O(n^{\log_b a}) \\ a < b^d & O(n^d) \\ a = b^d & O(n^d \log n) \end{cases}$$

• 分块 (分块直接计算): $A, B: n \times n$

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \quad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} \Rightarrow C = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}$$

$$\left. \begin{aligned} C_{11} &= A_{11} B_{11} + A_{12} B_{21} \\ C_{12} &= A_{11} B_{12} + A_{12} B_{22} \\ C_{21} &= A_{21} B_{11} + A_{22} B_{21} \\ C_{22} &= A_{21} B_{12} + A_{22} B_{22} \end{aligned} \right\} \begin{aligned} &\text{计算复杂度:} \\ &T(n) = 8T\left(\frac{n}{2}\right) + cn^2 \\ &\text{主项决定: } T(n) = O(n^{\log_2 8}) = O(n^3) \end{aligned}$$

③ Strassen 分块算法:

$$\left. \begin{aligned} P_1 &= A_{11}(B_{12} - B_{22}) \\ P_2 &= (A_{11} + A_{12})B_{22} \\ P_3 &= (A_{21} + A_{22})B_{11} \\ P_4 &= A_{22}(B_{21} - B_{11}) \\ P_5 &= (A_{11} + A_{22})(B_{11} + B_{22}) \\ P_6 &= (A_{12} - A_{22})(B_{21} + B_{22}) \\ P_7 &= (A_{11} - A_{21})(B_{11} + B_{12}) \end{aligned} \right\} \Rightarrow \begin{aligned} C_{11} &= P_5 + P_4 - P_2 + P_6 \\ C_{12} &= P_1 + P_2 \\ C_{21} &= P_3 + P_4 \\ C_{22} &= P_5 + P_1 - P_3 - P_7 \end{aligned}$$

hw: 验证 $C_{11} \dots C_{22}$ 正确性.

算法复杂度: $T(n) = 7T\left(\frac{n}{2}\right) + cn^2$

$$(\text{主项}) = O(\log_2 7) \approx O(n^{2.81})$$

• 参考文献:

[1] Strassen V. Gaussian elimination is not optimal. Numerische Mathematik, 1969, 13(4): 354-356.

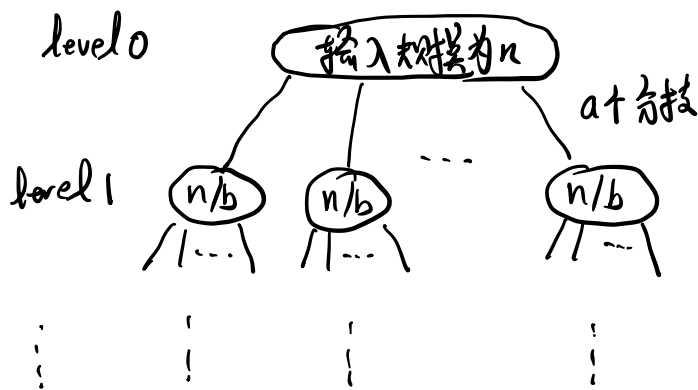
[2] ~. Discovering faster matrix multiplication algorithms with reinforcement learning. Nature, 2022.

LN.11. 算法复杂度分析 — 主方法:

def. 主方法: $T(n) = aT(\frac{n}{b}) + c \cdot n^d$, $T(1) = 1$

$$T(n) = \begin{cases} a > b^d & O(n^{\log_b a}) \\ a < b^d & O(n^d) \\ a = b^d & O(n^d \log n) \end{cases}$$

证明: 不失一般性, 设 $n = b^k$.



level k $\circ \dots \circ \dots \circ \dots \circ \dots \circ \dots \circ$ $\sum a^k = a^{\log_b n} \uparrow$
 $k = \log_b n$

对于 level j { 子树数: a^j
子树的大小: $\frac{n}{b^j}$

第 j 层合并: $T(\text{level } j) \leq a^j \cdot c \left(\frac{n}{b^j} \right)^d = c \cdot n^d \left(\frac{a}{b^d} \right)^j$

$\Rightarrow T(n) \leq c \cdot n^d \sum_{j=0}^k \left(\frac{a}{b^d} \right)^j$

理解 { a : 子树的扩散率
 b^d : 子树工作量的收缩率

① $a=b^d: T(n) = c \cdot n^d (\log_b^n + 1) = O(n^d \log n)$

②, ③: $r = \frac{a}{b^d}, r^0 + r^1 + \dots + r^k = \frac{r^{k+1} - 1}{r - 1}$

$\begin{cases} \text{若 } r > 1, \Rightarrow O(r^k) \\ \text{若 } r < 1, \Rightarrow O(1) \end{cases}$

②: $a < b^d: T(n) = O(n^d)$

③: $a > b^d: T(n) = c \cdot n^d \cdot \left(\frac{a}{b^d}\right)^{\log_b^n}$

利用换底公式:

$c \log_b^a = a \log_b^c$

$= c n^d \cdot n^{\log_b \frac{a}{b^d}}$

$= c n^d \cdot n^{(\log_b^a - d)}$

$= c n^{\log_b^a} = O(n^{\log_b^a})$

理解: $n^{\log_b^a} = a^{\log_b^n} = a^k = \omega(1)$

LN12. 找第k小元素:

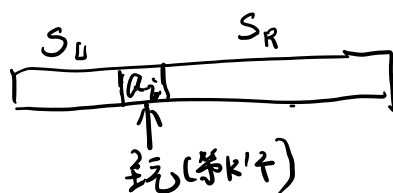
· 问题定义: 已知n个元素 $S = \{a_1, a_2, \dots, a_n\}$

求第k小元素.

· 朴素算法: 排序. 输出 $O(n \log n)$

· 分治算法:

问题: 快速排序:



$\begin{cases} k' < k: \text{在 } S_R \text{ 中, 递归 } S_L \\ k' > k: \text{在 } S_L \text{ 中, 递归 } S_R \end{cases}$

· 算法 $\text{Select}(S, k)$

① 选一个主元 a_i , 若 a_i 在第k个位置, 则返回 a_i

$$\textcircled{2} S_L = \{a_j \in S \mid a_j < a_i\}$$

$$S_R = \{a_j \in S \mid a_j \geq a_i\}$$

③ 若 $k > |S_L|$, 则 $\text{select}(S_R, k - |S_L| - 1)$

$$\mathbb{Z}_{2^k} \mid \text{select}(S_u, k)$$

• 观察: 主元心造样:

① 最佳: 中位法. 对于 n 个元素, 最坏情况下为 $T(n) = T(\frac{n}{2}) + cn = O(n \log n)$

② 比教好 10 元。



$$\frac{1}{4}|S| \leq \text{主元位置} \leq \frac{3}{4}|S|$$

归纳: $T(n) \leq T(\frac{3}{4}n) + cn = O(n)$

· 我主元:

① 随机主元^(*)

令随机变量 T 为随机变量. $\text{Prob}(T = n^{1/2}) > 0$

$$E(T) = O(n)$$

Thm. $E(T) = O(n)$.

pf. $E(T) = \sum_j \text{Prob}(T=j) \cdot (\text{第 } j \text{ 次等待量})$

$$\text{prob}(\text{好手玩}) = \frac{1}{2}$$

定义一个 phase (阶段): 不好 $\rightarrow$ 好气

$\left. \begin{array}{c} \text{---} \\ \text{---} \end{array} \right\} \begin{array}{c} x \\ x \end{array} \right\} \text{phase 0, } T_0, \leq n$

$$\begin{matrix} \checkmark \\ \times \\ \checkmark \end{matrix} \} \text{phase 1, } T_1, \leq \frac{3}{4}n$$

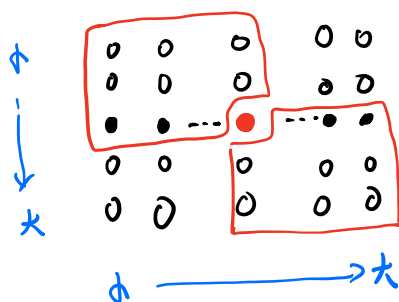
- { i) phase i 开始时数组长度 $\leq (\frac{3}{4})^i \cdot n$
 ii) 在每个 phase, $E(\text{比较次数}) = 2$

$$E(T) = \sum_{i=0}^{\infty} T_i \cdot \text{比较次数}$$

$$= \sum_{i=0}^{\infty} (\frac{3}{4})^i \cdot n \cdot 2$$

$$= 2n \cdot \sum_{i=0}^{\infty} (\frac{3}{4})^i = 8n = O(n)$$

② = 次取中找元:



$$T(n) \leq O(n) + T(\frac{n}{5}) + T(\frac{3}{4}n)$$

对 $\frac{n}{5}$ 组,
组内排序

对 $\frac{n}{5}$ 个中位数
发第 $\frac{n}{10}$ 个元素

剩下元素.

5个元素排序. 是 $\frac{n}{5}$ 组

Thm. $T(n) = T(\frac{n}{5}) + T(\frac{3}{4}n) + cn \leq dn$ d 为某个常数

Pf. $T(n) \leq d \cdot \frac{n}{5} + d \cdot \frac{3}{4}n + cn \leq dn$

$\Rightarrow d \geq 20c$. 就成立. (为子问题内证)

归纳: $T(n) = O(n)$

Q1: 7个为一组. 递归表达式 = ?

Q2: 5, 7, 9, 11, 13. 为一组.

① 递归表达式

② 用模拟验证. 观察对于什么范围的 n . 使用那种分组最好.

* 算法复杂度分析.

1) 递归: $T(n) \leq cn + T(C_1n) + T(C_2n) + \dots + T(C_kn)$

$$C_1 + C_2 + \dots + C_k < 1$$

则 $T(n) \leq dn$

$$d \geq \frac{c}{1 - (C_1 + \dots + C_k)}$$

2) 迭代: $T(n)$ 估算 $\left\{ \begin{array}{l} \text{① 直接代入, 递归求解.} \\ \text{② 主方法} \\ \text{③ 假设成立, 进行验证.} \end{array} \right.$